

Aula 12

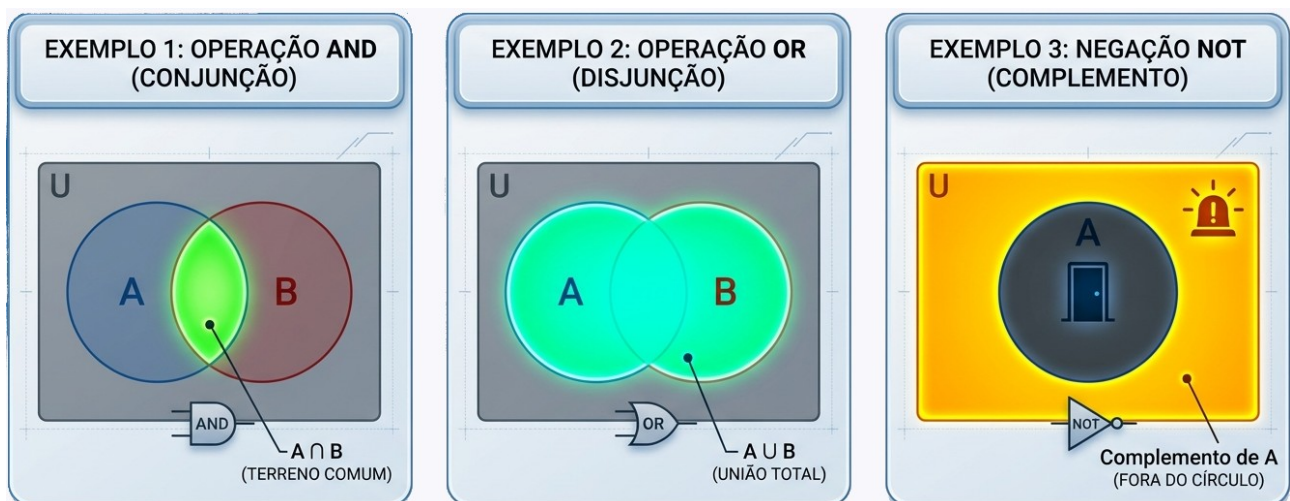
Representações de Funções Booleanas Computacionais

Para projetar circuitos ou entender o fluxo de dados em um software, utilizamos diferentes representações das funções booleanas. Vamos explorar as três principais:

1. Diagramas de Venn e Círculos de Euler

Essas representações utilizam a teoria dos conjuntos para visualizar as relações lógicas. O Diagrama de Venn mostra todas as interseções possíveis entre as variáveis, enquanto o de Euler foca apenas nas relações existentes.

- **Exemplo 1** (Operação AND): Imagine dois círculos, A e B. A área onde eles se sobrepõem representa a conjunção (A and B). É o "terreno comum" onde ambas as condições são verdadeiras simultaneamente.
- **Exemplo 2** (Operação OR): A união total das áreas de A e B representa a disjunção (A or B). Se o elemento está em qualquer um dos círculos (ou em ambos), a condição é satisfeita.
- **Exemplo 3** (Negação NOT): Se temos o conjunto A, a área fora do círculo (o "complemento") representa a negação de A. Em um sistema de segurança, se "Porta Fechada" é o círculo, a área externa é o estado de alerta "Porta Aberta".



2. Tabelas-Verdade

Esta é a representação tabular e exaustiva. Ela lista todas as combinações possíveis de entradas (0 ou 1) e o resultado final da função. É a ferramenta favorita para validar a equivalência entre dois algoritmos.

- **Exemplo 1** (Porta XOR - Ou Exclusivo): Útil para sistemas de paridade. A saída é 1 apenas se as entradas forem diferentes.

A	B	Saída
0	0	0
0	1	1
1	0	1
1	1	0

- **Exemplo 2** (Implicação Lógica): Se P então Q. Só é falsa se a premissa for verdadeira e a conclusão falsa. É a base das estruturas **if-then** na programação.
- **Exemplo 3** (NAND): A negação da conjunção. É conhecida como "porta universal" na arquitetura de computadores, pois qualquer outra função pode ser construída apenas com portas NAND.

A operação NAND (do inglês Not AND) é simplesmente a negação de um AND.

Ou seja:

- AND (E): só é verdadeiro quando todas as entradas são verdadeiras
- NAND: faz o contrário → só é falso quando todas são verdadeiras

A	B	A AND B	A NAND B
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

Exemplo em programação

Em muitas linguagens (como C, Java, Python), não existe um operador direto para NAND, mas podemos construí-lo assim (os exemplos a seguir utilizam Python):

```
def nand(a, b):  
    return not (a and b)  
  
print(nand(True, True))    # False  
print(nand(True, False))   # True
```

Por que é chamada de “porta universal”? Porque você consegue montar todas as outras operações lógicas usando apenas NAND.

1. NOT usando NAND

Negar um valor:

```
def NOT(a):  
    return nand(a, a)
```

Explicação:

Se $a \text{ AND } a = a$

Então $\text{NAND}(a, a) = \text{NOT}(a)$

2. AND usando NAND

```
def AND(a, b):  
    return NOT(nand(a, b))
```

Ou direto:

```
def AND(a, b):  
    return nand(nand(a, b), nand(a, b))
```

3. OR usando NAND

```
def OR(a, b):  
    return nand(NOT(a), NOT(b))
```

Exemplo prático (programação)

Imagine a validação de um usuário no sistema, onde o usuário só pode acessar se estiver **logado E tiver permissão**.

```
logado = True
tem_permissao = True

acesso = not (logado and tem_permissao) # NAND

if acesso:
    print("Acesso NEGADO")
else:
    print("Acesso permitido")
```

Aqui o NAND está sendo usado para detectar **quando a condição ideal NÃO acontece**.

Conclui-se então que a operação NAND é poderosa porque:

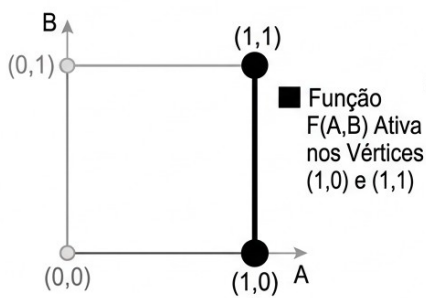
- Combina duas operações em uma só: AND + NOT
- Permite reconstruir qualquer lógica possível
- Por isso, em hardware real (circuitos), muitas vezes se usa só NAND para montar tudo

3. Representação Geométrica (n-cubos)

Esta é uma visão mais avançada e espacial. Representamos as n variáveis como vértices de uma figura geométrica. Para duas variáveis, temos um quadrado; para três, um cubo.

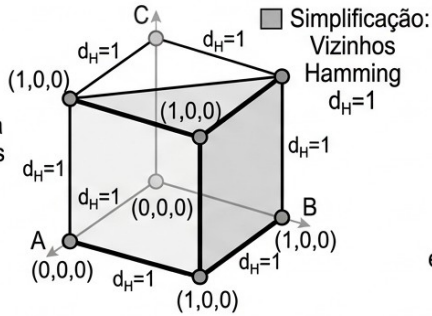
- **Exemplo 1 (O Quadrado Unitário):** Com duas variáveis (A e B), os vértices são $(0,0)$, $(0,1)$, $(1,0)$ e $(1,1)$. Se uma função booleana ativa os vértices $(1,0)$ e $(1,1)$, podemos visualizar geometricamente que a variável B não influencia o resultado naquela face, simplificando a lógica.
- **Exemplo 2 (O Cubo de 3 Variáveis):** Para uma função com três entradas, usamos os 8 vértices de um cubo. Vizinhos que diferem em apenas um bit (distância de Hamming igual a 1) são fundamentais para métodos de simplificação como o Mapa de Karnaugh.
- **Exemplo 3 (Hiper-cubos):** Em sistemas complexos com 4 ou mais variáveis, entramos na geometria de n -dimensões. Embora difícil de desenhar, matematicamente permite que o computador otimize rotas de dados minimizando a distância entre estados lógicos.

1. O Quadrado Unitário (2 Variáveis: A, B)



Simplificação: $F(A,B) = A$
(B não influencia)

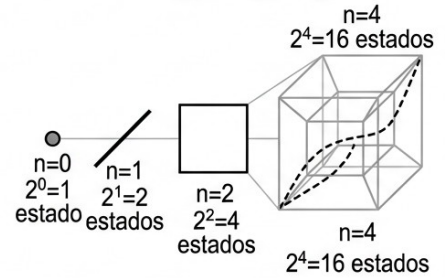
2. O Cubo de 3 Variáveis (A, B, C)



Simplificação:
Vizinhos Hamming $d_H=1$

3. Hipercubos (4+ Variáveis)

Hipercubos e Otimização de Rotas de Dados



Minimização de distância entre estados lógicos para rotas de dados complexas.

Nota de Aula: Lembrem-se que, embora pareçam temas distintos, todos dizem a mesma coisa. Uma linha na tabela-verdade corresponde a um ponto no cubo e a uma região específica no diagrama de Venn.